# 共通API仕様書

**作成日**: 2026-01-21 16:30
**最終更新**: 2026-01-21 16:30
**バージョン**: 1.0.0
**対象**: 実験管理者メニュー全体
**総API数**: 2個
**実装状況**: 0個実装済み、2個未実装

---

## 📊 実装状況サマリー

| カテゴリ | 実装済み | 未実装 | 合計 |
|---|---|---|---|
| ファイル操作 | 0 | 2 | 2 |
| **合計** | **0** | **2** | **2** |

**完了率**: 0% (0/2)

---

## 🎯 API一覧

| # | API名 | メソッド | エンドポイント | 状態 | 優先度 |
|---|---|---|---|---|---|
| 1 | ファイルアップロード | POST | `/api/v1/files/upload` | 🔴 未実装 | ⭐⭐⭐ 高 |
| 2 | ファイルダウンロード | GET | `/api/v1/files/:fileId/download` | 🔴 未実装 | ⭐⭐⭐ 高 |

---

## 📋 概要

共通APIは、プロジェクト管理とフォーマット管理の両方で使用されるファイル操作APIを提供します。

### 主な特徴

1. **contextパラメータによる保存先制御**
   - `project`: プロジェクトフォルダに保存

- `format`: フォーマットフォルダに保存

2. **マルチテナント分離**
   - すべてのファイルパスに `org_{organization_prefix}` プレフィックス
3. **署名付きURL**
   - ダウンロード時はPre-signed URLを生成
   - 有効期限: 1時間

---

# 🔐 共通仕様

## 認証・認可

**認証方式**: JWT + Session Hybrid

**必須ヘッダー**:

```
Authorization: Bearer maiml_session-{sessionId}_{timestamp}
```

**organization_id の取得**:

- ログイン時に `X-Organization-Id` ヘッダーから組織IDを取得し、セッションに保存します
- ログイン後のAPIでは、`Authorization` ヘッダーのセッショントークンから自動的に取得されます
- リクエストヘッダーで `X-Organization-Id` を指定する必要はありません
- `authMiddleware` がセッション情報を検証し、ユーザー情報（organization_idを含む）を `req.user` に格納します

**権限要件**:

- すべてのAPI: `experiment_admin` または `system_admin` ロール必須

**ミドルウェアチェーン**:

```
router.METHOD('/path',
  authMiddleware,           // 1. JWT認証（セッション検証、req.userに格納）
  permissionCheck([...]),   // 2. 権限チェック
  controller.handler        // 3. コントローラー処理
);
```

**ミドルウェアの動作**:

1. `authMiddleware`: JWTトークン（`maiml_session-{sessionId}_{timestamp}`）を検証し、セッション情報を取得して`req.user`に格納（organizationIdを含む）
2. `permissionCheck`: ユーザーのロール（`req.user.role`）を検証
3. コントローラー: `req.user.organizationId`を使用してテナント分離されたデータにアクセス

**マルチテナント分離の実現**:

- ログイン時にJWTトークンに`organizationId`を含める
- セッション中に組織は変わらない（組織切り替えは再ログインが必要）
- 各コントローラーで`req.user.organizationId`を使用してDBクエリをフィルタ
- S3パスには`org_{organization_prefix}`を強制的に付与

## マルチテナント分離

- すべてのS3パスに`org_{organization_prefix}`プレフィックス必須
- contextパラメータで保存先を判定

## 統一レスポンス形式

**成功レスポンス**:

```
{
  "success": true,
  "data": { ... },
```

```
  "meta": {
    "timestamp": "2026-01-21T16:00:00Z",
    "requestId": "req_abc123"
  }
}
```

**エラーレスポンス**:

```
{
  "success": false,
  "error": {
    "code": "ERROR_CODE",
    "message": "エラーメッセージ",
    "details": []
  },
  "meta": {
    "timestamp": "2026-01-21T16:00:00Z",
    "requestId": "req_abc123"
  }
}
```

---

# 📋 API詳細仕様

## 1. ファイルアップロード 🔴

**エンドポイント**: `POST /api/v1/files/upload`

**説明**: プロジェクトまたはフォーマットにファイルをアップロード

**認証**: 必須

**権限**: `experiment_admin`, `system_admin`

**優先度**: ⭐⭐⭐ 高

### リクエスト

**Content-Type**: `multipart/form-data`

**フォームデータ**:

| フィールド | 型 | 必須 | 説明 |
|---|---|---|---|
| `file` | File | ○ | アップロードするファイル |
| `context` | string | ○ | コンテキスト（`project` または `format`） |
| `contextId` | UUID | ○ | プロジェクトIDまたはフォーマットID |
| `filePath` | string | × | 相対ファイルパス（指定しない場合はファイル名のみ） |
| `overwrite` | boolean | × | 上書き許可（デフォルト: false） |

## S3パス生成ロジック

### プロジェクトの場合 (context=project):

```
org_{organization_prefix}/projects/{project_name}/{filePath || filename}
```

### フォーマットの場合 (context=format):

```
org_{organization_prefix}/formats/{format_prefix}/{filePath || filename}
```

## バリデーション

| 項目 | 制約 |
|---|---|
| ファイルサイズ | 最大100MB |
| ファイル形式 | `.maiml`, `.xml`, `.csv`, `.json`, `.txt`, `.pdf`, `.png`, `.jpg` |
| contextId | 有効なプロジェクトIDまたはフォーマットID |
| filePath | 不正なパス（`..`, 絶対パス）を禁止 |

## レスポンス

### 成功時 (201 Created):

```
{
  "success": true,
  "data": {
    "file_id": "file-uuid",
    "file_name": "sample.maiml",
    "file_path": "org_maiml_lab_001/projects/Sample_Project/sample.maiml",
```

```
    "file_size": 12345,
    "file_type": "maiml",
    "context": "project",
    "context_id": "project-uuid",
    "uploaded_at": "2026-01-21T16:00:00Z",
    "uploaded_by": "user-uuid"
  },
  "meta": {
    "timestamp": "2026-01-21T16:00:00Z",
    "requestId": "req_abc123"
  }
}
```

**エラー時**:

- 400: バリデーションエラー（ファイルサイズ超過、不正なファイル形式、不正なパス）
- 401: 認証エラー
- 403: 権限不足
- 404: contextIdが存在しない
- 409: ファイルが既に存在（overwrite=falseの場合）
- 413: ファイルサイズが大きすぎる（Payload Too Large）
- 500: システムエラー

## 実装要件

**バックエンド処理フロー**:

1. マルチパートデータをパース
2. contextIdの存在確認（projectまたはformat）
3. organization_prefixの取得
4. S3キーの生成
5. ファイルの存在確認（overwriteチェック）
6. S3へアップロード
7. file_metadataテーブルに登録
8. レスポンス返却

**使用サービス**:

```
import { getStorageService } from '@/services/storageService';

const storageService = getStorageService();
await storageService.uploadFile(
  organizationPrefix,
  s3Key,
  fileBuffer,
  { contentType: file.mimetype }
);
```

### 使用例

**プロジェクトへのアップロード**:

```
curl -X POST http://localhost:3001/api/v1/files/upload \
  -H "Authorization: Bearer maiml_session-abc123_1234567890" \
  -H "X-Organization-Id: org-uuid" \
  -F "file=@sample.maiml" \
  -F "context=project" \
  -F "contextId=550e8400-e29b-41d4-a716-446655440000" \
  -F "overwrite=false"
```

**フォーマットへのアップロード（サブフォルダ指定）**:

```
curl -X POST http://localhost:3001/api/v1/files/upload \
  -H "Authorization: Bearer maiml_session-abc123_1234567890" \
  -H "X-Organization-Id: org-uuid" \
  -F "file=@workflow.maiml" \
  -F "context=format" \
  -F "contextId=format-uuid" \
  -F "filePath=chemistry/synthesis/workflow.maiml"
```

---

## 2. ファイル/フォルダダウンロード 🔴

**エンドポイント**: `POST /api/v1/files/download`

**説明**: S3パス（ファイルまたはフォルダ）のダウンロード

**認証**: 必須

**権限**: `experiment_admin`, `system_admin`

**優先度**: ⭐⭐⭐ 高

## リクエストボディ

**Content-Type**: `application/json`

**パラメータ**:

| パラメータ | 型 | 必須 | デフォルト | 説明 |
|---|---|---|---|---|
| `path` | string | ○ | - | S3パス（相対パス、organization_prefix不要） |
| `expires` | integer | × | 3600 | 有効期限（秒、ファイルのみ、最大86400=24時間） |

**リクエスト例（ファイル）**:

```
{
  "path": "projects/Sample_Project/sample.maiml",
  "expires": 3600
}
```

**リクエスト例（フォルダ）**:

```
{
  "path": "projects/Sample_Project"
}
```

**パスの形式**:

- ファイル: `projects/Sample_Project/sample.maiml`
- フォルダ: `projects/Sample_Project` または `projects/Sample_Project/`
- **重要**: organization_prefixは含めない（サーバー側で自動付与）

## ファイル/フォルダの判定方法

1. **パスの末尾で判定**

- 末尾が `/` の場合 → フォルダとして扱う
- それ以外 → ファイルとして扱う

2. **file_metadataで検証**
   - ファイルとして扱う場合: 完全一致で存在確認
   - フォルダとして扱う場合: プレフィックス一致で存在確認

## レスポンス

**ファイルの場合（署名付きURL）**

**Status**: 200 OK


```
{
  "success": true,
  "data": {
    "type": "file",
    "file_name": "sample.maiml",
    "file_path": "org_maiml_lab_001/projects/Sample_Project/sample.maiml",
    "file_size": 12345,
    "download_url":
"https://s3.amazonaws.com/bucket/org_maiml_lab_001/projects/Sample_Project/sample.maiml?X-Amz-
Algorithm=...",
    "expires_at": "2026-01-21T17:00:00Z",
    "expires_in": 3600
  },
  "meta": {
    "timestamp": "2026-01-21T16:00:00Z",
    "requestId": "req_abc123"
  }
}
```


**フォルダの場合（ZIP直接ダウンロード）**

**Status**: 200 OK

**Content-Type**: `application/zip`

**Content-Disposition**: `attachment; filename="Sample_Project.zip"`

**レスポンスボディ**: ZIPファイルのバイナリデータ（ストリーミング）

**注意**: フォルダの場合、JSONレスポンスではなく、直接ZIPファイルをストリーミングで返します。

## エラーレスポンス

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_INVALID_PATH",
    "message": "不正なパスが指定されています",
    "details": ["パスに'..'が含まれています"]
  },
  "meta": {
    "timestamp": "2026-01-21T16:00:00Z",
    "requestId": "req_abc123"
  }
}
```

**エラーコード**:

- 400: `VALIDATION_INVALID_PATH` - 不正なパス
- 400: `VALIDATION_PATH_TOO_LONG` - パスが長すぎる
- 400: `FOLDER_TOO_LARGE` - フォルダのサイズが制限を超過（最大1GB）
- 400: `FOLDER_TOO_MANY_FILES` - ファイル数が制限を超過（最大1,000ファイル）
- 401: 認証エラー
- 403: 権限不足（他組織のファイル）
- 404: `RESOURCE_NOT_FOUND` - ファイル/フォルダが存在しない
- 404: `FOLDER_EMPTY` - フォルダにファイルが存在しない
- 500: システムエラー

## フォルダダウンロードの制限

1. **最大ZIPサイズ**: 1GB（圧縮後）
2. **最大ファイル数**: 1,000ファイル
3. **タイムアウト**: 5分
4. **同時ダウンロード数**: ユーザーあたり3並列まで

## 実装要件

**バックエンド処理フロー（ファイル）**:

1. リクエストボディからpathを取得
2. パスのバリデーション（セキュリティチェック）
3. organization_prefixを付与してフルパスを構築
4. file_metadataでファイルの存在確認
5. organization_idの一致確認（マルチテナント分離）
6. S3署名付きURLを生成
7. JSONレスポンスを返却

**バックエンド処理フロー（フォルダ）**:

1. リクエストボディからpathを取得
2. パスのバリデーション（セキュリティチェック）
3. organization_prefixを付与してプレフィックスを構築
4. S3プレフィックス検索でファイル一覧を取得
5. ファイル数・サイズのチェック
6. archiverでZIPをストリーミング生成
7. ZIPファイルをレスポンスとして返却

**使用サービス**:

```
import { getStorageService } from '@/services/storageService';

const storageService = getStorageService();

// ファイルの場合
const signedUrl = await storageService.getSignedUrl(
  organizationPrefix,
  filePath,
  expiresIn
);

// フォルダの場合
const files = await storageService.listFiles(
  organizationPrefix,
  folderPrefix  // プレフィックス検索
);
```

## S3プレフィックス検索（フォルダダウンロード）

**S3構造例**:

```
org_maiml_lab_001/projects/Sample_Project/file1.maiml
org_maiml_lab_001/projects/Sample_Project/file2.maiml
org_maiml_lab_001/projects/Sample_Project/subdir/file3.maiml
```

**プレフィックス検索**:

- `projects/Sample_Project` をプレフィックスとして検索
- プレフィックスに一致するすべてのキーを取得（再帰的）
- サブフォルダも含めて取得される

**ZIP生成実装例**:

```
import archiver from 'archiver';

// ZIPアーカイブを作成（ストリーミング）
const archive = archiver('zip', {
  zlib: { level: 9 } // 最高圧縮
});

// レスポンスヘッダーを設定
res.setHeader('Content-Type', 'application/zip');
res.setHeader('Content-Disposition', `attachment; filename="${folderName}.zip"`);

// ZIPをレスポンスにパイプ
archive.pipe(res);

// 各ファイルをZIPに追加
for (const file of files) {
  const fileContent = await storageService.downloadFile(
    organizationPrefix,
    file.key
  );
  archive.append(fileContent, { name: relativePath });
}

await archive.finalize();
```

## セキュリティ考慮事項

### 1. パストラバーサル攻撃の防止

```
function validatePath(path: string): void {
  if (path.startsWith('/')) {
    throw new ValidationError('絶対パスは使用できません');
  }
  if (path.includes('..')) {
    throw new ValidationError('不正なパスが含まれています');
  }
  if (path.includes('\0')) {
    throw new ValidationError('不正な文字が含まれています');
  }
  if (path.startsWith('org_')) {
    throw new ValidationError('パスにorganization_prefixを含めないでください');
  }
}
```

### 2. マルチテナント分離の検証

```
// file_metadataのorganization_idとリクエストユーザーのorganization_idを照合
if (fileMetadata.organization_id !== req.user.organizationId) {
  throw new AuthorizationError('このファイルへのアクセス権限がありません');
}
```

### 3. organization_prefix強制付与

```
function buildFullPath(
  organizationPrefix: string,
  requestedPath: string
): string {
  // 必ずorganization_prefixで始まることを保証
  return `${organizationPrefix}/${requestedPath}`;
}
```

### 4. 署名付きURLの有効期限（ファイルのみ）

- デフォルト: 1時間（3600秒）
- 最大: 24時間（86400秒）
- 最小: 60秒

## 使用例

### ファイルダウンロード:

```
curl -X POST http://localhost:3001/api/v1/files/download \
  -H "Authorization: Bearer maiml_session-abc123_1234567890" \
  -H "X-Organization-Id: org-uuid" \
  -H "Content-Type: application/json" \
  -d '{
    "path": "projects/Sample_Project/sample.maiml",
    "expires": 3600
  }'
```

### フォルダダウンロード（ZIP）:

```
curl -X POST http://localhost:3001/api/v1/files/download \
  -H "Authorization: Bearer maiml_session-abc123_1234567890" \
  -H "X-Organization-Id: org-uuid" \
  -H "Content-Type: application/json" \
  -d '{
    "path": "projects/Sample_Project"
  }' \
  --output Sample_Project.zip
```

### フロントエンドでの使用例（ファイル）:

```
// APIから署名付きURLを取得
const response = await apiClient.post('/files/download', {
  path: 'projects/Sample_Project/sample.maiml',
  expires: 3600
});

const { download_url } = response.data;

// ブラウザでダウンロードを開始
const link = document.createElement('a');
link.href = download_url;
link.download = response.data.file_name;
link.click();
```

### フロントエンドでの使用例（フォルダ）:

```
// ZIP直接ダウンロード
const response = await apiClient.post('/files/download', {
  path: 'projects/Sample_Project'
}, {
  responseType: 'blob'  // バイナリデータとして受信
});

// BlobからダウンロードURL作成
const url = window.URL.createObjectURL(response.data);
const link = document.createElement('a');
link.href = url;
link.download = 'Sample_Project.zip';
link.click();
window.URL.revokeObjectURL(url);
```

---

# 🔒 セキュリティ考慮事項

## ファイルアップロード

**セキュリティチェック**:

- [ ] ファイルサイズ制限（最大100MB）
- [ ] 許可されたファイル形式のみ受付
- [ ] パストラバーサル攻撃の防止（`..`, 絶対パス禁止）
- [ ] organization_prefixの強制付与
- [ ] contextIdの存在確認と権限検証

**実装例**:

```
// パストラバーサル攻撃の防止
const sanitizePath = (path: string): string => {
  // 絶対パスを禁止
  if (path.startsWith('/')) {
    throw new ValidationError('絶対パスは使用できません');
  }

  // '..'を禁止
  if (path.includes('..')) {
    throw new ValidationError('不正なパスが含まれています');
```

```
  }

  // パスの正規化
  return path.replace(/\/+/g, '/').trim();
};
```

## ファイルダウンロード

**セキュリティチェック**:

- [ ] organization_idの一致確認（マルチテナント分離）
- [ ] 署名付きURLの有効期限設定
- [ ] ファイルパスの検証

**実装例**:

```
// マルチテナント分離の検証
const fileMetadata = await db
  .selectFrom('file_metadata')
  .where('file_id', '=', fileId)
  .where('organization_id', '=', organizationId)  // 必須
  .selectAll()
  .executeTakeFirst();

if (!fileMetadata) {
  throw new NotFoundError('ファイルが見つかりません');
}
```

---

# 🔧 実装ガイドライン

## ファイルメタデータ管理

**file_metadataテーブル構造**:

```
CREATE TABLE file_metadata (
  file_id UUID PRIMARY KEY,
  organization_id UUID NOT NULL,
```

```
  file_name VARCHAR(255) NOT NULL,
  file_path VARCHAR(1000) NOT NULL,
  file_size BIGINT NOT NULL,
  file_type VARCHAR(50),
  context VARCHAR(50) NOT NULL,  -- 'project' or 'format'
  context_id UUID NOT NULL,
  is_archive BOOLEAN DEFAULT FALSE,
  version INTEGER,
  created_at TIMESTAMP NOT NULL,
  updated_at TIMESTAMP NOT NULL,
  created_by UUID NOT NULL,
  FOREIGN KEY (organization_id) REFERENCES organization(organization_id),
  CONSTRAINT check_context CHECK (context IN ('project', 'format'))
);

CREATE INDEX idx_file_metadata_org_context ON file_metadata(organization_id, context,
context_id);
```

## storageService.ts の活用

**推奨パターン**:

```
import { getStorageService } from '@/services/storageService';

const storageService = getStorageService();

// アップロード
await storageService.uploadFile(
  organizationPrefix,
  filePath,
  fileBuffer,
  { contentType: mimeType }
);

// ダウンロード用URL生成
const signedUrl = await storageService.getSignedUrl(
  organizationPrefix,
  filePath,
  expiresIn
);

// ファイル削除
await storageService.deleteFile(
  organizationPrefix,
```

```
  filePath
);
```

## エラーハンドリング

**統一されたエラーレスポンス**:

```
// ファイルサイズ超過
{
  "code": "VALIDATION_FILE_TOO_LARGE",
  "message": "ファイルサイズが大きすぎます（最大100MB）"
}

// 不正なファイル形式
{
  "code": "VALIDATION_INVALID_FILE_TYPE",
  "message": "このファイル形式はサポートされていません"
}

// 権限不足
{
  "code": "PERMISSION_INSUFFICIENT",
  "message": "このファイルへのアクセス権限がありません"
}
```

---

# 📚 関連ドキュメント

## 規約

- API設計ポリシー
- バックエンド規約
- セキュリティ規約

## 設計書

- API一覧（統合版）
- API実装方針

- S3構造
- プロジェクト管理API
- フォーマット管理API

---

# 📝 変更履歴

| バージョン | 日付 | 変更内容 | 担当者 |
|---|---|---|---|
| 1.0.0 | 2026-01-21 | 新規作成 | Cline |

---

**作成者**: Cline
**作成日**: 2026-01-21 16:30
**最終更新**: 2026-01-21 16:30